

Program 1: Design and Implementation of a Full Adder Using VHDL

```
Library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity full_adder is
```

```
Port (
```

```
    A    : in    STD_LOGIC;
```

```
    B    : in    STD_LOGIC;
```

```
    Cin   : in    STD_LOGIC;
```

```
    Sum   : out   STD_LOGIC;
```

```
    Cout  : out   STD_LOGIC
```

```
);
```

```
end full_adder;
```

```
architecture Behavioral of full_adder is
```

```
begin
```

```
    Sum <= A XOR B XOR Cin;  -- Sum is 1 for odd number of 1s
```

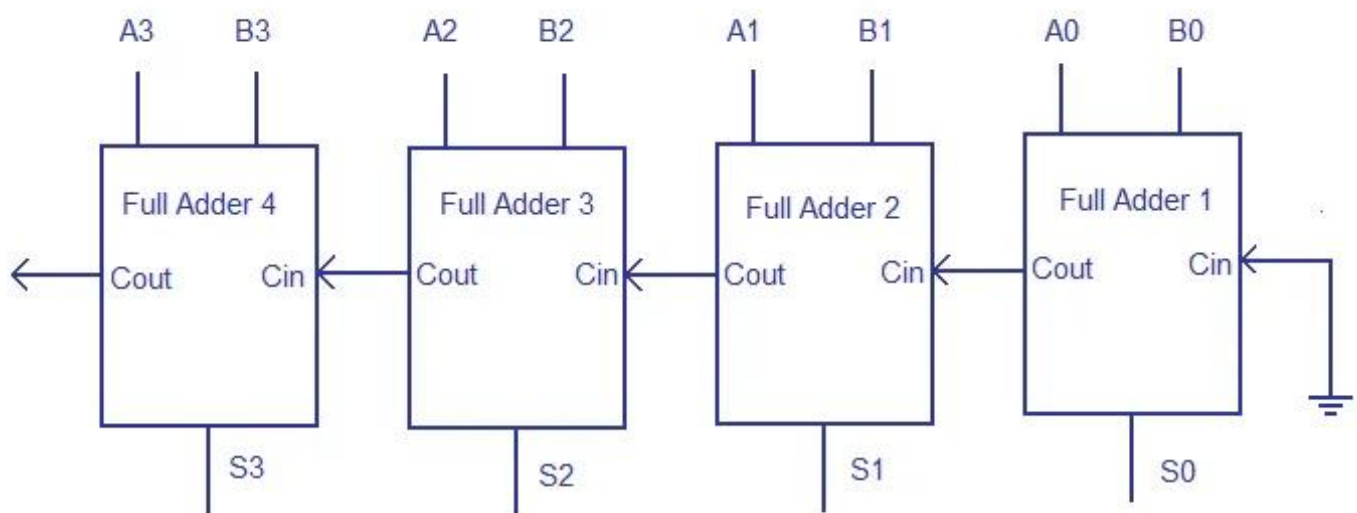
```
    Cout <= (A AND B) OR (Cin AND (A XOR B));  -- Carry when at least  
                                                two inputs are 1
```

```
end Behavioral;
```

Truth Table

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Figure 1: Block Diagram of 4-Bit Ripple Carry Adder Using Four Full Adders



4 bit ripple carry adder

www.circuitstoday.com

Program 2: Design and Implementation of a 4-Bit Ripple Carry Adder Using VHDL

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity rca_4bit is
    Port (
        A    : in  STD_LOGIC_VECTOR(3 downto 0);
        B    : in  STD_LOGIC_VECTOR(3 downto 0);
        Cin   : in  STD_LOGIC;
        Sum   : out STD_LOGIC_VECTOR(3 downto 0);
        Cout  : out STD_LOGIC
    );
end rca_4bit;

architecture Structural of rca_4bit is

    component full_adder
        Port (
            A    : in  STD_LOGIC;
            B    : in  STD_LOGIC;
            Cin   : in  STD_LOGIC;
            Sum   : out STD_LOGIC;
            Cout  : out STD_LOGIC
        );
    end component;

    signal C : STD_LOGIC_VECTOR(3 downto 0); -- internal carry chain

begin
    -- four full adders chained: carry-out of each feeds carry-in of next

    FA0: full_adder port map(A(0), B(0), Cin, Sum(0), C(0));
    FA1: full_adder port map(A(1), B(1), C(0), Sum(1), C(1));
    FA2: full_adder port map(A(2), B(2), C(1), Sum(2), C(2));
    FA3: full_adder port map(A(3), B(3), C(2), Sum(3), Cout);

end Structural;
```

Program 3: Simulation and Verification of a 4-Bit Ripple Carry Adder Using Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_rca_4bit is      -- testbench, no ports
end tb_rca_4bit;

architecture Test of tb_rca_4bit is
    signal A, B : STD_LOGIC_VECTOR(3 downto 0);
    signal Cin   : STD_LOGIC;
    signal Sum   : STD_LOGIC_VECTOR(3 downto 0);
    signal Cout  : STD_LOGIC;
begin
    -- device under test
    DUT: entity work.rca_4bit
        port map (
            A    => A,
            B    => B,
            Cin   => Cin,
            Sum   => Sum,
            Cout  => Cout
        );

    -- apply test inputs
    process
    begin
        A <= "0101"; B <= "0011"; Cin <= '0';      -- 5 + 3 = 8
        wait for 10 ns;
        A <= "1111"; B <= "0001"; Cin <= '0';      -- 15 + 1 = 16 (overflow)
        wait for 10 ns;
        A <= "1010"; B <= "0101"; Cin <= '0';      -- 10 + 5 = 15
        wait for 10 ns;
        A <= "1111"; B <= "1111"; Cin <= '0';      -- 15 + 15 = 30 (overflow)
        wait for 10 ns;
        A <= "1010"; B <= "0011"; Cin <= '1';      -- 10 + 3 + 1 = 14
        wait for 10 ns;
        wait;
    end process;
end Test;
```

Figure 2: Simulation Waveform of 4-Bit Ripple Carry Adder in ModelSim

